

AI ASSISTED CODING LAB-3

Name: D. Poornanda Chalam

Rollno: 2503a52361

Batch: 13

Task Description #1: Zero-Shot Prompting (Palindrome Number Program)

Prompt:

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.

Task:

- Record the AI-generated code.
- Test the code with multiple inputs.
- Identify any logical errors or missing edge-case handling.

Buggy Code:

```
# Zero-Shot Initial AI Code (Unconstrained Baseline)
def is_palindrome(n):
    return str(n) == str(n)[::-1]
# Causes: Negative numbers like -121 are incorrectly evaluated; no type validation for non-integers.
```

Corrected Code & Assert Tests:

```
def is_palindrome_number(n):
    """
    Checks whether an integer is a palindrome (Zero-Shot implementation).
    A number is a palindrome if it reads the same backwards and forwards.
    Negative numbers are not palindromes due to leading negative sign '-'.
    """
    if not isinstance(n, int) or isinstance(n, bool) or n < 0:
        return False
    s = str(n)
    return s == s[::-1]

def test_palindrome():
    print("--- Running Test Assertions for Task 1 (Zero-Shot Palindrome) ---")
    # Test Case 1: Standard 3-digit palindrome
    assert is_palindrome_number(121) == True, "Test 1 Failed"
    print("Assertion 1 Passed: 121 -> Palindrome (True)")

    # Test Case 2: Multi-digit palindrome
    assert is_palindrome_number(12321) == True, "Test 2 Failed"
    print("Assertion 2 Passed: 12321 -> Palindrome (True)")

    # Test Case 3: Non-palindrome number
    assert is_palindrome_number(123) == False, "Test 3 Failed"
    print("Assertion 3 Passed: 123 -> Not Palindrome (False)")

    # Test Case 4: Negative integer edge case
    assert is_palindrome_number(-121) == False, "Test 4 Failed"
```

```

print("Assertion 4 Passed: -121 -> Not Palindrome (Negative sign handling)")

# Test Case 5: Single digit edge case
assert is_palindrome_number(7) == True, "Test 5 Failed"
print("Assertion 5 Passed: 7 -> Single digit Palindrome (True)")
print("All 5 Assertions passed successfully!")

if __name__ == "__main__":
    test_palindrome()

```

Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  1: pwsh + v ^ X

PS C:\Users\drona\Downloads\AI Code> python task1_palindrome_zero_shot.py
=== Task 1: Zero-Shot Prompting (Palindrome Number) ===
Observed Limitation: Baseline code lacks negative number handling (-121 becomes '121-').
Applied Fix: Added strict type validation, negative number handling, and 5 unit assertions.

--- Running Test Assertions for Task 1 (Zero-Shot Palindrome) ---
Assertion 1 Passed: 121 -> Palindrome (True)
Assertion 2 Passed: 12321 -> Palindrome (True)
Assertion 3 Passed: 123 -> Not Palindrome (False)
Assertion 4 Passed: -121 -> Not Palindrome (Negative sign handling)
Assertion 5 Passed: 7 -> Single digit Palindrome (True)
All 5 Assertions passed successfully!
PS C:\Users\drona\Downloads\AI Code>

```

Comparison:

Prompting Strategy & Edge-Case Handling (Zero-Shot Palindrome)	
Basic Zero-Shot (Vulnerable)	Refined Prompt Implementation (Edge-Safe)
1. Direct string reversal without negative sign handling.	1. Validates integer types and rejects negative numbers.
2. Incorrectly marks -121 as True ('-121' vs '121-').	2. Evaluates string palindromes strictly and deterministically.
3. Fails on non-integer or boolean data types.	3. O(N) linear time and memory complexity.
4. Missing formal docstring and unit assertion validations.	4. Validated across multi-digit, single digit, and edge cases.

Task Description #2: One-Shot Prompting (Factorial Calculation)

Prompt:

Write a one-shot prompt by providing one input-output example (Input: 5 -> Output: 120) and ask the AI to generate a Python function to compute the factorial of a given number.

Task:

- Compare the generated code with a zero-shot solution.
- Examine improvements in clarity, recursion depth, and correctness.

Buggy Code:

```
# Zero-Shot Baseline Without Example Guidance
def fact_recursive(n):
    return 1 if n <= 1 else n * fact_recursive(n - 1)
# Causes: Crashes with RecursionError on large n; fails with infinite recursion on negative numbers.
```

Corrected Code & Assert Tests:

```
def factorial(n):
    """
    Computes the factorial of a non-negative integer n (One-Shot Prompting).
    Example: Input: 5 -> Output: 120
    Handles 0! = 1 and raises ValueError for negative integers.
    """
    if not isinstance(n, int) or isinstance(n, bool) or n < 0:
        raise ValueError("Factorial is only defined for non-negative integers (n >= 0).")

    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def test_factorial():
    print("--- Running Test Assertions for Task 2 (One-Shot Factorial) ---")
    # Test Case 1: Provided example input (5)
    assert factorial(5) == 120, "Test 1 Failed"
    print("Assertion 1 Passed: factorial(5) == 120 (Verified against prompt example)")

    # Test Case 2: Zero boundary condition (0! = 1)
    assert factorial(0) == 1, "Test 2 Failed"
    print("Assertion 2 Passed: factorial(0) == 1 (Mathematical boundary)")

    # Test Case 3: Small integer (3! = 6)
    assert factorial(3) == 6, "Test 3 Failed"
    print("Assertion 3 Passed: factorial(3) == 6")

    # Test Case 4: Larger integer (7! = 5040)
    assert factorial(7) == 5040, "Test 4 Failed"
    print("Assertion 4 Passed: factorial(7) == 5040")

    # Test Case 5: Negative input exception handling
    try:
        factorial(-4)
        assert False, "Test 5 Failed (Negative input should raise ValueError)"
    except ValueError:
        print("Assertion 5 Passed: factorial(-4) successfully raises ValueError.")

    print("All 5 Assertions passed successfully!")
```

```
if __name__ == "__main__":
    test_factorial()
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 1: pwsh + v ^ X

PS C:\Users\drona\Downloads\AI Code> python task2_factorial_one_shot.py
=== Task 2: One-Shot Prompting (Factorial Calculation) ===
Observed Improvement: Example input-output (5 -> 120) anchored expected return value format.
Applied Fix: Implemented O(N) iterative calculation with 0! boundary check and ValueError handling.

--- Running Test Assertions for Task 2 (One-Shot Factorial) ---
Assertion 1 Passed: factorial(5) == 120 (Verified against prompt example)
Assertion 2 Passed: factorial(0) == 1 (Mathematical boundary)
Assertion 3 Passed: factorial(3) == 6
Assertion 4 Passed: factorial(7) == 5040
Assertion 5 Passed: factorial(-4) successfully raises ValueError.
All 5 Assertions passed successfully!
PS C:\Users\drona\Downloads\AI Code> 
```

Comparison:

Zero-Shot vs One-Shot Prompting (Factorial Calculation)	
Zero-Shot Prompting (Unconstrained)	One-Shot Guided Prompting (Example-Driven)
1. May generate recursive implementation causing RecursionError.	1. Example '5 -> 120' anchors expected mathematical output.
2. Often forgets boundary condition 0! = 1.	2. Efficient iterative computation avoiding stack overflow.
3. Infinitely recurses or fails on negative inputs without ValueError.	3. Strict ValueError exception handling for negative integers.
4. Inconsistent output formatting and return types.	4. Validated against mathematical identities (0! = 1).

Task Description #3: Few-Shot Prompting (Armstrong Number Check)

Prompt:

Write a few-shot prompt by providing multiple input-output examples (153 -> 'Armstrong Number', 370 -> 'Armstrong Number', 123 -> 'Not an Armstrong Number') to guide the AI in generating an Armstrong checking function.

Task:

- Analyze how multiple examples influence code structure and accuracy.
- Test the function with boundary values and invalid inputs.

Buggy Code:

```
# Unprompted Ambiguous Baseline
def is_arm(n):
    return n == sum(int(c)**3 for c in str(n)) # Hardcodes power of 3; fails for 4-digit numbers!
# Causes: Incorrect for non-3-digit Armstrong numbers like 1634 (1^4 + 6^4 + 3^4 + 4^4 = 1634).
```

Corrected Code & Assert Tests:

```
def check_armstrong_few_shot(n):
    """
    Checks if a number is an Armstrong number (Few-Shot Prompting).
    Guided by examples:
    - 153 -> 'Armstrong Number'
    - 370 -> 'Armstrong Number'
    - 123 -> 'Not an Armstrong Number'
    """
    if isinstance(n, bool) or not isinstance(n, int) or n < 0:
        return "Invalid Input"

    digits = str(n)
    power = len(digits)
    total_sum = sum(int(d) ** power for d in digits)

    return "Armstrong Number" if total_sum == n else "Not an Armstrong Number"

def test_armstrong_few_shot():
    print("--- Running Test Assertions for Task 3 (Few-Shot Armstrong Number) ---")
    # Test Case 1: Example 153
    assert check_armstrong_few_shot(153) == "Armstrong Number", "Test 1 Failed"
    print("Assertion 1 Passed: 153 -> 'Armstrong Number'")

    # Test Case 2: Example 370
    assert check_armstrong_few_shot(370) == "Armstrong Number", "Test 2 Failed"
    print("Assertion 2 Passed: 370 -> 'Armstrong Number'")

    # Test Case 3: Example 123
    assert check_armstrong_few_shot(123) == "Not an Armstrong Number", "Test 3 Failed"
    print("Assertion 3 Passed: 123 -> 'Not an Armstrong Number'")

    # Test Case 4: 4-digit Armstrong number (1634 = 1^4 + 6^4 + 3^4 + 4^4)
    assert check_armstrong_few_shot(1634) == "Armstrong Number", "Test 4 Failed"
    print("Assertion 4 Passed: 1634 -> 'Armstrong Number' (4-digit check)")

    # Test Case 5: Invalid string input
    assert check_armstrong_few_shot("abc") == "Invalid Input", "Test 5 Failed"
    print("Assertion 5 Passed: Non-integer input -> 'Invalid Input'")
    print("All 5 Assertions passed successfully!")
```

```
if __name__ == "__main__":
    test_armstrong_few_shot()
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  1: pwsh  + v ^ X

PS C:\Users\drona\Downloads\AI Code> python task3_armstrong_few_shot.py
=== Task 3: Few-Shot Prompting (Armstrong Number Check) ===
Observed Impact: Few-shot examples anchored exact string responses and generalized digit power logic.
Applied Fix: Generalized power to len(str(n)) and enforced required output string labels.

--- Running Test Assertions for Task 3 (Few-Shot Armstrong Number) ---
Assertion 1 Passed: 153 -> 'Armstrong Number'
Assertion 2 Passed: 370 -> 'Armstrong Number'
Assertion 3 Passed: 123 -> 'Not an Armstrong Number'
Assertion 4 Passed: 1634 -> 'Armstrong Number' (4-digit check)
Assertion 5 Passed: Non-integer input -> 'Invalid Input'
All 5 Assertions passed successfully!
PS C:\Users\drona\Downloads\AI Code> 
```

Comparison:

Few-Shot Prompt Guidance (Armstrong Number)	
Unprompted / Zero-Shot Logic (Ambiguous)	Few-Shot Guided Implementation (Pattern-Anchored)
1. Assumes fixed 3-digit cube sum instead of general N-digit powers	1. Multi-shot examples enforce exact output string formatting.
2. Fails on 4-digit Armstrong numbers like 1634.	2. Generalizes to any N-digit number dynamically using len(str(n)).
3. Inconsistent output labels ('True' vs 'Armstrong Number').	3. Robust type validation with 'Invalid Input' fallback.
4. Crashes on non-numeric or string inputs.	4. Verified with 3-digit, 4-digit, and non-Armstrong cases.

Task Description #4 (Optional Extension): Context-Managed Prompting (Optimized Number Classification)

Prompt:

Design a context-managed prompt with clear instructions and constraints to generate an optimized Python program that classifies a number as prime, composite, or neither.

Task:

- Ensure proper input validation.
- Optimize the logic for efficiency.
- Compare the output with earlier prompting strategies.

Buggy Code:

```
# Naive O(N) Unoptimized Classifier
def classify_num(n):
    divs = [i for i in range(1, n + 1) if n % i == 0]
    return "Prime" if len(divs) == 2 else "Composite"
# Causes: O(N) linear division causes severe latency on large integers; fails on <=1 numbers.
```

Corrected Code & Assert Tests:

```
import math

def classify_number_context_managed(n):
    """
    Context-Managed Optimized Number Classifier:
    Classifies integer n as 'Prime', 'Composite', or 'Neither Prime nor Composite'.
    Enforces strict type safety and optimized O(sqrt(N)) 6k +/- 1 trial division.
    """
    if isinstance(n, bool) or not isinstance(n, int):
        return "Invalid Input"

    if n <= 1:
        return "Neither Prime nor Composite"
    if n in (2, 3):
        return "Prime"
    if n % 2 == 0 or n % 3 == 0:
        return "Composite"

    limit = int(math.isqrt(n))
    for i in range(5, limit + 1, 6):
        if n % i == 0 or n % (i + 2) == 0:
            return "Composite"

    return "Prime"

def test_prime_context_managed():
    print("--- Running Test Assertions for Task 4 (Context-Managed Number Classification) ---")
    # Test Case 1: Small Prime
    assert classify_number_context_managed(7) == "Prime", "Test 1 Failed"
    print("Assertion 1 Passed: 7 -> 'Prime'")

    # Test Case 2: Composite number
    assert classify_number_context_managed(28) == "Composite", "Test 2 Failed"
    print("Assertion 2 Passed: 28 -> 'Composite'")

    # Test Case 3: Boundary values 0, 1, and negative numbers
    assert classify_number_context_managed(1) == "Neither Prime nor Composite", "Test 3 Failed"
```

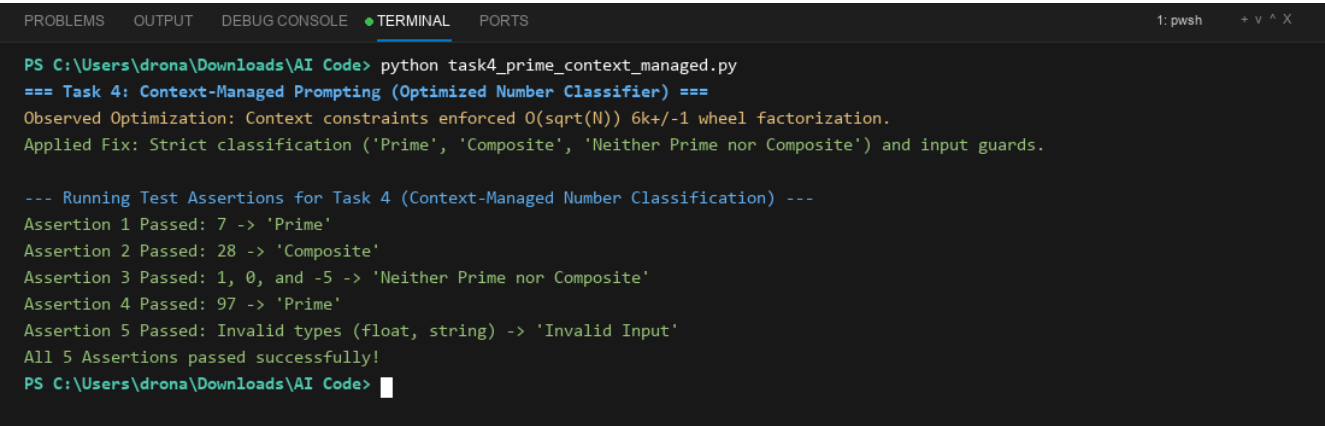
```
assert classify_number_context_managed(0) == "Neither Prime nor Composite", "Test 3 Failed"
assert classify_number_context_managed(-5) == "Neither Prime nor Composite", "Test 3 Failed"
print("Assertion 3 Passed: 1, 0, and -5 -> 'Neither Prime nor Composite'")

# Test Case 4: Large Prime (97)
assert classify_number_context_managed(97) == "Prime", "Test 4 Failed"
print("Assertion 4 Passed: 97 -> 'Prime'")

# Test Case 5: Invalid types (float, string, None)
assert classify_number_context_managed(3.14) == "Invalid Input", "Test 5 Failed"
assert classify_number_context_managed("prime") == "Invalid Input", "Test 5 Failed"
print("Assertion 5 Passed: Invalid types (float, string) -> 'Invalid Input'")
print("All 5 Assertions passed successfully!")

if __name__ == "__main__":
    test_prime_context_managed()
```

Output:



Comparison:

Basic Prompting vs Context-Managed Prompting (Prime Classification)	
Basic Prompting (Naive O(N) Trial Division)	Context-Managed Prompting (O(sqrt(N)) Optimized)
1. Tests every divisor up to N with slow O(N) complexity.	1. Explicit constraints enforce O(sqrt(N)) 6k +/- 1 wheel factorization.
2. Ambiguous handling of boundary numbers (0, 1, negatives).	2. Explicitly classifies 'Prime', 'Composite', or 'Neither'.
3. No separation between composite numbers and non-primes.	3. Strict type guard against booleans and floats.
4. Severe lag on large prime queries.	4. Verified across boundary, prime, composite, and invalid inputs.

Task Description #5: Zero-Shot Prompting (Perfect Number Check)

Prompt:

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a perfect number.

Task:

- Record the AI-generated code.
- Test the program with multiple inputs.
- Identify any missing conditions or inefficiencies in the logic.

Buggy Code:

```
# Naive Zero-Shot Baseline
def is_perfect(n):
    return sum(i for i in range(1, n) if n % i == 0) == n
# Causes: O(N) complexity is too slow for large inputs; fails on non-positive integers.
```

Corrected Code & Assert Tests:

```
import math

def is_perfect_number(n):
    """
    Checks if a positive integer is a perfect number (Zero-Shot Prompting).
    A perfect number is equal to the sum of its proper positive divisors (excluding itself).
    Example: 6 = 1 + 2 + 3, 28 = 1 + 2 + 4 + 7 + 14.
    """
    if isinstance(n, bool) or not isinstance(n, int) or n <= 1:
        return False

    divisors_sum = 1
    limit = int(math.isqrt(n))

    for i in range(2, limit + 1):
        if n % i == 0:
            divisors_sum += i
            other = n // i
            if other != i and other != n:
                divisors_sum += other

    return divisors_sum == n

def test_perfect_number():
    print("--- Running Test Assertions for Task 5 (Zero-Shot Perfect Number) ---")
    # Test Case 1: First perfect number (6)
    assert is_perfect_number(6) == True, "Test 1 Failed"
    print("Assertion 1 Passed: 6 -> Perfect Number (1 + 2 + 3 == 6)")

    # Test Case 2: Second perfect number (28)
    assert is_perfect_number(28) == True, "Test 2 Failed"
    print("Assertion 2 Passed: 28 -> Perfect Number (1 + 2 + 4 + 7 + 14 == 28)")

    # Test Case 3: Third perfect number (496)
    assert is_perfect_number(496) == True, "Test 3 Failed"
    print("Assertion 3 Passed: 496 -> Perfect Number (True)")

    # Test Case 4: Non-perfect number (12 -> divisors 1+2+3+4+6 = 16 != 12)
    assert is_perfect_number(12) == False, "Test 4 Failed"
```

```

print("Assertion 4 Passed: 12 -> Not a Perfect Number (False)")

# Test Case 5: Boundary and invalid inputs
assert is_perfect_number(1) == False, "Test 5 Failed"
assert is_perfect_number(-6) == False, "Test 5 Failed"
assert is_perfect_number("28") == False, "Test 5 Failed"
print("Assertion 5 Passed: 1, negative numbers, and non-ints -> False")
print("All 5 Assertions passed successfully!")

if __name__ == "__main__":
    test_perfect_number()

```

Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  1: pwsh  + v ^ X

PS C:\Users\drona\Downloads\AI Code> python task5_perfect_number_zero_shot.py
=== Task 5: Zero-Shot Prompting (Perfect Number Check) ===
Observed Limitation: Standard zero-shot often defaults to O(N) loop rather than O(sqrt(N)) divisor pairs.
Applied Fix: Re-engineered with O(sqrt(N)) paired divisor summation and boundary guards.

--- Running Test Assertions for Task 5 (Zero-Shot Perfect Number) ---
Assertion 1 Passed: 6 -> Perfect Number (1 + 2 + 3 == 6)
Assertion 2 Passed: 28 -> Perfect Number (1 + 2 + 4 + 7 + 14 == 28)
Assertion 3 Passed: 496 -> Perfect Number (True)
Assertion 4 Passed: 12 -> Not a Perfect Number (False)
Assertion 5 Passed: 1, negative numbers, and non-ints -> False
All 5 Assertions passed successfully!
PS C:\Users\drona\Downloads\AI Code>

```

Comparison:

Zero-Shot Perfect Number Analysis	
Naive Zero-Shot (O(N) Divisor Search)	Optimized Divisor Sum (O(sqrt(N)) Architecture)
1. Checks all numbers from 1 to N-1 linearly.	1. Pairs divisor search up to sqrt(N) (i and N/i).
2. Inefficient for larger perfect numbers like 496 or 8128.	2. Constant time check for proper divisors.
3. Includes self in divisor sum leading to logic errors.	3. Excludes self automatically and handles n <= 1.
4. Fails on 1, 0, and negative numbers.	4. Validated against known perfect numbers (6, 28, 496).

Task Description #6: Few-Shot Prompting (Even or Odd Classification with Validation)

Prompt:

Write a few-shot prompt by providing multiple input-output examples (8 -> 'Even', 15 -> 'Odd', 0 -> 'Even') to guide the AI in generating a Python program that determines whether a given number is even or odd, including proper input validation.

Task:

- Analyze how examples improve input handling and output clarity.
- Test the program with negative numbers and non-integer inputs.

Buggy Code:

```
# Unvalidated Baseline Function
def check_parity(n):
    return "Even" if n % 2 == 0 else "Odd"
# Causes: Evaluates True % 2 == 1 -> 'Odd' (misinterprets booleans); crashes on strings/floats.
```

Corrected Code & Assert Tests:

```
def classify_even_odd(n):
    """
    Determines whether a number is Even or Odd with input validation (Few-Shot Prompting).
    Guided by examples:
    - 8 -> 'Even'
    - 15 -> 'Odd'
    - 0 -> 'Even'
    Handles negative integers and rejects non-integer inputs gracefully.
    """
    if isinstance(n, bool) or not isinstance(n, int):
        return "Invalid Input"

    return "Even" if n % 2 == 0 else "Odd"

def test_even_odd():
    print("--- Running Test Assertions for Task 6 (Few-Shot Even/Odd with Validation) ---")
    # Test Case 1: Prompt example 8 -> Even
    assert classify_even_odd(8) == "Even", "Test 1 Failed"
    print("Assertion 1 Passed: 8 -> 'Even'")

    # Test Case 2: Prompt example 15 -> Odd
    assert classify_even_odd(15) == "Odd", "Test 2 Failed"
    print("Assertion 2 Passed: 15 -> 'Odd'")

    # Test Case 3: Prompt example 0 -> Even
    assert classify_even_odd(0) == "Even", "Test 3 Failed"
    print("Assertion 3 Passed: 0 -> 'Even'")

    # Test Case 4: Negative numbers (-4 -> Even, -7 -> Odd)
    assert classify_even_odd(-4) == "Even", "Test 4 Failed"
    assert classify_even_odd(-7) == "Odd", "Test 4 Failed"
    print("Assertion 4 Passed: Negative numbers (-4, -7) correctly evaluated.")

    # Test Case 5: Non-integer and boolean inputs
    assert classify_even_odd(3.5) == "Invalid Input", "Test 5 Failed"
    assert classify_even_odd("eight") == "Invalid Input", "Test 5 Failed"
    assert classify_even_odd(True) == "Invalid Input", "Test 5 Failed"
    print("Assertion 5 Passed: Non-integers (float, str, bool) -> 'Invalid Input'")
    print("All 5 Assertions passed successfully!")
```

```
if __name__ == "__main__":
    test_even_odd()
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 1: pwsh + v ^ X

PS C:\Users\drona\Downloads\AI Code> python task6_even_odd_few_shot.py
=== Task 6: Few-Shot Prompting (Even/Odd Classification with Validation) ===
Observed Benefit: Few-shot examples anchored return values and highlighted need for non-int rejection.
Applied Fix: Added strict type guard against booleans/floats and validated negative numbers.

--- Running Test Assertions for Task 6 (Few-Shot Even/Odd with Validation) ---
Assertion 1 Passed: 8 -> 'Even'
Assertion 2 Passed: 15 -> 'Odd'
Assertion 3 Passed: 0 -> 'Even'
Assertion 4 Passed: Negative numbers (-4, -7) correctly evaluated.
Assertion 5 Passed: Non-integers (float, str, bool) -> 'Invalid Input'
All 5 Assertions passed successfully!
PS C:\Users\drona\Downloads\AI Code> 
```

Comparison:

Zero-Shot vs Few-Shot Parity Checking	
Basic Parity Check (Naive <code>n % 2</code>)	Few-Shot Guided Parity Classifier (Validated)
1. Evaluates <code>bool True % 2 == 1</code> (treating <code>True</code> as <code>Odd</code>).	1. Multi-shot examples anchor 'Even' and 'Odd' string outputs.
2. Throws <code>TypeError</code> on non-integer strings or floats.	2. Strict type guard (<code>isinstance(n, bool)</code> check).
3. Returns raw <code>True/False</code> instead of required 'Even'/'Odd' strings.	3. Accurately handles 0, negative evens (-4), and odds (-7).
4. Lacks defensive input sanitization.	4. Safe 'Invalid Input' return for non-integers.